

编 号	题 号	标 题	知 识 点	代 码 作 者
例 5-1	UVa 1401	Remember the Word	Trie	刘汝佳
例 5-2	POJ 1961	Period	KMP: 失配函数	刘汝佳
例 5-3	UVa 1449	Dominating Patterns	AC 自动机	刘汝佳
例 5-4	UVa 11468	Substring	AC 自动机	刘汝佳、陈锋

5.2 后缀数组、Hash和Manacher

虽然因为后缀自动机以及Hash的存在，后缀数组的应用范围已经大大缩小，但是本节仍然提供了一些题目。另外，Hash是一种特别容易编写且可以解决子串以及后缀问题的方法，Manacher也是解决回文串相关问题的不错选择。

例5-5 【后缀数组】生命的形式 (Life Forms, UVa 11107)

输入 n ($1 \leq n \leq 100$) 个DNA序列，你的任务是求出一个长度最大的字符串，使得它在超过一半的DNA序列中连续出现。如果有多解，按照字典序从小到大输出所有解。

【代码实现】

```

// 刘汝佳
#include <algorithm>
#include <cstdio>
#include <cstring>

using namespace std;
#define _for(i, a, b) for (int i = (a); i < (b); ++i)
#define _rep(i, a, b) for (int i = (a); i <= (b); ++i)
typedef long long LL;

template <int SZ>
struct SuffixArray {
    int s[SZ];           // 原始字符数组 (最后一个字符必须是 0, 前面的字符必须非 0)
    int sa[SZ];           // 后缀数组
    int rank[SZ];         // 名次数组, rank[0]一定是 n-1, 即最后一个字符
    int height[SZ];       // height 数组
    int t[SZ], t2[SZ], c[SZ]; // 辅助数组
    int n;                // 字符个数

    void clear() { n = 0; fill_n(sa, SZ, 0); }

    // m 为最大字符值加 1, 调用之前需设置好 s 和 n
    void build_sa(int m) {
        int i, *x = t, *y = t2;
        for (i = 0; i < m; i++) c[i] = 0;
        for (i = 0; i < n; i++) c[x[i] = s[i]]++;
        for (i = 1; i < m; i++) c[i] += c[i - 1];
        for (i = n - 1; i >= 0; i--) sa[--c[x[i]]] = i;
        for (int k = 1; k <= n; k <= 1) {
            int p = 0;
            for (i = n - k; i < n; i++) y[p++] = i;
            for (i = 0; i < n; i++)
                if (sa[i] >= k) y[p++] = sa[i] - k;
            for (i = 0; i < m; i++) c[i] = 0;
        }
    }
};

```

```

    for (i = 0; i < n; i++) c[x[y[i]]]++;
    for (i = 0; i < m; i++) c[i] += c[i - 1];
    for (i = n - 1; i >= 0; i--) sa[--c[x[y[i]]]] = y[i];
    swap(x, y), p = 1, x[sa[0]] = 0;
    for (i = 1; i < n; i++)
        x[sa[i]] = y[sa[i - 1]] == y[sa[i]] && y[sa[i - 1] + k] == y[sa[i] + k]
            ? p - 1 : p++;
    if (p >= n) break;
    m = p;
}
}

void build_height() {
    for (int i = 0; i < n; i++) rank[sa[i]] = i;
    for (int i = 0, k = 0; i < n; i++) {
        if (k) k--;
        int j = sa[rank[i] - 1];
        while (s[i + k] == s[j + k]) k++;
        height[rank[i]] = k;
    }
}

};

const int MAXL = 1000 + 8, MAXN = 100 + 4;
int idx[MAXL * MAXN], flag[MAXN], N;
char buf[MAXL];
SuffixArray<MAXL * MAXN> sa;

bool good(int L, int R) {
    if (R - L <= N / 2) return false;
    fill_n(flag, MAXN, 0);
    int cnt = 0;
    _for(i, L, R) {
        int x = idx[sa.sa[i]];
        if (x != N && !flag[x]) flag[x] = 1, cnt++;
    }
    return cnt > N / 2;
}

void print_sub(int L, int R) { // print s[L,R]
    _for(i, L, R) printf("%c", sa.s[i] - 1 + 'a');
    puts("");
}

bool print_sol(int len, bool print = false) {
    for (int L = 0, R = 1; R <= sa.n; R++) {
        if (R == sa.n || sa.height[R] < len) { // 新开一段
            if (good(L, R)) {
                if (!print) return true;
            }
        }
    }
}

```

```

        print_sub(sa.sa[L], sa.sa[L] + len);
    }
    L = R;
}
}
return false;
}

void solve(int maxLen) {
    if (!print_sol(1)) {
        puts("?");
        return;
    }
    int L = 1, R = maxLen, M;
    while (L < R) {
        M = L + (R - L + 1) / 2;
        if (print_sol(M))
            L = M;
        else
            R = M - 1;
    }
    print_sol(L, true);
}

// 给字符串加上一个字符, 属于字符串 i
void add(int ch, int i) { idx[sa.n] = i, sa.s[sa.n++] = ch; }

int main() {
    for (int t = 0; scanf("%d", &N) == 1 && N; t++) {
        if (t) puts("");
        int maxl = 0;
        sa.n = 0;
        _for(i, 0, N) {
            scanf("%s", buf);
            int sz = strlen(buf);
            maxl = max(maxl, sz);
            _for(j, 0, sz) add(buf[j] - 'a' + 1, i);
            add(100 + i, N);
        }
        add(0, N);
        if (N == 1)
            puts(buf);
        else
            sa.build_sa(N + 100), sa.build_height(), solve(maxl);
    }
    return 0;
}
/*

```

算法分析请参考:《算法竞赛入门经典——训练指南》升级版 3.4.2 节例题 19

注意：要理解后缀数组的排序部分，要首先理解基数排序（Radix Sort）

同类题目：Alice's Classified Message, HDU 5558

*/

例5-6 【基于Hash的LCP】口吃的外星人（Stammering Aliens, SWERC 2009, UVa 12206）

有一个口吃的外星人，说的话里包含很多重复的字符串，比如 babab 包含两个 bab。给定这个外星人说的一句话，找出至少出现 m 次的最长字符串。

【代码实现】

```

// 刘汝佳
#include <algorithm>
#include <cstdio>
#include <iostream>
using namespace std;
#define _for(i, a, b) for (int i = (a); i < (b); ++i)
typedef long long LL;
typedef unsigned long long ULL;
const int MAXN = 40000 + 8;
const ULL x = 123;
ULL H[MAXN], PX[MAXN], Hash[MAXN];
int N, sa[MAXN];
void init_PX() { // 初始化 Hash, 计算所用的  $x^p$ 
    PX[0] = 1;
    _for(i, 1, MAXN) PX[i] = x * PX[i - 1];
}

void init_hash(const string& s) {
    N = s.length();
    H[N] = 0;
    for (int i = N - 1; i >= 0; i--) H[i] = (s[i] - 'a' + 1) + H[i + 1] * x;
}

bool hash_cmp(int a, int b) {
    if (Hash[a] != Hash[b]) return Hash[a] < Hash[b];
    return a < b;
}

bool ok(int L, int M, int& pos) { // 是否有长度至少为 len 的子串出现 M 次以上
    _for(i, 0, N - L + 1) sa[i] = i, Hash[i] = H[i] - H[i + L] * PX[L];
    sort(sa, sa + N - L + 1, hash_cmp);
    pos = -1;
    int c = 0;
    _for(i, 0, N - L + 1) {
        if (i == 0 || Hash[sa[i]] != Hash[sa[i - 1]]) c = 0;
        if (++c >= M) pos = max(pos, sa[i]);
    }
    return pos >= 0;
}

```

```

int main() {
    ios::sync_with_stdio(false), cin.tie(0);
    init_PX();
    string word;
    for (int t = 0, pos, M; cin >> M >> word && M; t++) {
        init_hash(word);
        if (!ok(1, M, pos)) {
            puts("none");
            continue;
        }
        int l = 1, r = N + 1;
        while (l + 1 < r) {
            int m = l + (r - l) / 2;
            if (ok(m, M, pos)) l = m;
            else r = m;
        }
        ok(1, M, pos);
        printf("%d %d\n", l, pos);
    }
    return 0;
}

```

/*

算法分析请参考：《算法竞赛入门经典——训练指南》升级版 3.4.3 节例题 20

注意：在二分查找之前要首先判断问题是否有解

同类题目：A Horrible Poem, POI 2012, 牛客 NC 50316

*/

例5-7 【manacher】扩展成回文（Extend to Palindrome, UVa 11475）

给出一个仅由大小写英文字母组成的长度为 N ($N \leq 10^5$) 的字符串 S ，请在 S 后面增加最少的字符，使其形成一个回文串。

【代码实现】

```

// 陈锋
#include <bits/stdc++.h>
using namespace std;
const int MAXN = 1e5 + 4;
char S[MAXN], T[MAXN * 2];
int P[MAXN * 2];
void manacher(const char *s, int len) {
    int l = 0;
    T[l++] = '$', T[l++] = '#';
    for (int i = 0; i < len; i++) T[l++] = s[i], T[l++] = '#';
    T[l] = 0;
    int r = 0, c = 0;
    for (int i = 0; i < l; i++) {
        int &p = P[i];
        p = r > i ? min(P[2 * c - i], r - i) : 1;
        while (T[i + p] == T[i - p]) p++;
        if (i + p > r) r = i + p, c = i;
    }
}
int main() {
    while (scanf("%s", S) == 1) {
        int ans = 0, L = strlen(S);
        manacher(S, L);
        for (int i = 0; i < 2 * L + 2; i++)
            if (P[i] + i == 2 * L + 2)
                ans = max(ans, P[i] - 1); // 此回文串是作为后缀出现的，更新答案
        printf("%s", S);
        for (int i = L - ans - 1; i >= 0; i--) printf("%c", S[i]);
        puts("");
    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 3.4.4 节例题 21
注意：manacher 算法插入的特殊字符可以根据不同的题目要求进行修改
同类题目：吉哥系列故事——完美队形 II，HDU 4513
        3-Palindromes, Codechef PALIN3
        Number of Palindromes, SPOJ NUMOFPAL
        Mirror Strings!!!, SPOJ MSUBSTR
*/

```

本节例题列表

本节讲解的例题及其囊括的知识点，如表5-2所示。